

Advanced Lesson 15 - Exercises:

Contexts

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: What is the current context?

`$Context`

Under a fresh kernel this is "Global` ". A context is a namespace: every symbol's full name is its context followed by a backquote followed by its short name, so a symbol typed as `x` is really `Global`x`. The backquote is the context mark and is not an apostrophe.

2: What is the current context path?

`$ContextPath`

The context path is the list of contexts searched, in order, when a short name is typed. It always contains "System`", where every built-in lives, and "Global`". Anything a loaded package has added appears here too - which is how Needs makes a package's exported names available without qualification.

3: A symbol in the current context

```
Clear[newSymbol];  
newSymbol = "Current";  
{Context[newSymbol], newSymbol}
```

The symbol was created in whatever context was current, so its context is `Global`` and its full name is `Global`newSymbol`.

4: The same short name in a different context

The obvious way to do this is to switch context with `Begin`, assign, and switch back with `End`. `Begin` does change the current context, and `End` restores the previous one and returns the name of the context being left. But the obvious way does not work here, and it is worth watching it fail before repairing it - the failure is exactly what Exercise 6 is about.

```
Begin["newContext`"];
newSymbol = "New";
End[]
```

```
{Context[newSymbol], newSymbol, NameQ["newContext`newSymbol"]}
```

What went wrong. The symbol is still `Global`newSymbol`, its value is now "New", and no symbol called `newContext`newSymbol` was ever created. `Begin` did switch the current context, but creating a symbol is not the first thing Mathematica tries when it reads a short name - it first searches the context path, and `Global`` is on that path. `Global`newSymbol` already existed from Exercise 3, so the short name resolved to it and the assignment overwrote it. Nothing new was created because nothing new was needed.

The repair below restores the `Global`` symbol and creates the other one by its full name, which bypasses the search entirely.

```
newSymbol = "Current";
newContext`newSymbol = "New";
{newSymbol, newContext`newSymbol}
```

Now the two symbols genuinely coexist. They share a short name and nothing else - different full names, different values, no interference. That is the entire purpose of the mechanism: two pieces of code can each have a symbol called `newSymbol` without either knowing about the other.

Real packages avoid the collision a third way. They call `Begin["`Private`"]` inside a `BeginPackage` block, which removes `Global`` from the context path for the duration, so short names inside the package cannot reach outside it.

5: Information on both symbols

```
?Global`newSymbol
```

```
?newContext`newSymbol
```

Each reports its own value: "Current" for the one in Global` and "New" for the one in newContext`. Asking about the short name alone would report whichever the context path finds first, which is why the full names are used here.

6: Why Remove, and not Clear or ClearAll

```
Clear[newSymbol];
{Context[newSymbol], ValueQ[newSymbol], NameQ["Global`newSymbol"]}
```

Clear removes a symbol's values, and ClearAll removes its values, definitions, attributes, messages and defaults. Neither removes the symbol. The cell above shows the difference: after Clear, the symbol has no value, but it still exists - it is still known to Global`, and NameQ still finds it.

That residue is what causes the trouble the exercise is asking about. Suppose Global`newSymbol already exists and the current context path includes Global`. Typing the short name newSymbol anywhere - including inside a Begin block for another context - resolves to the existing Global` symbol, because a symbol that already exists in a context on the path is found before a new one is created. No amount of clearing changes that, since clearing empties the symbol without unregistering the name. Remove is the operation that actually deletes the name from its context. Once Global`newSymbol has been removed, the next use of the short name has nothing to find and creates a fresh symbol in whichever context is current. That is why accessing "the same symbol" in a different context requires Remove: the obstruction is the existence of the name, not the presence of a value, and only Remove addresses existence.

```
Remove["Global`newSymbol"];
NameQ["Global`newSymbol"]
```

False - the name is gone. The symbol in newContext` is untouched, since it was never the one in the way.